

Smart Traffic Monitoring & Accident Prediction

Design Document – Bayesian Network (PS12)

BITS Pilani – MTech AI & ML | Group ID: G028

1. Solution Overview

The system is modeled using two binary Bayesian Networks and exact probabilistic inference. **Scenario 1** uses Road Accident (A) as the common cause of Traffic Delay (D) and Emergency Call (E). **Scenario 2** uses Traffic Signal Failure (S) as the common cause of Camera Alert (C) and Sensor Alert (R). The Python implementation reads all probabilities from `inputPS12.txt` and uses reusable complete joint enumeration; no query result or probability is hard-coded.

Exact inference principle: For a complete assignment x , the joint probability is the product of local conditional probabilities $P(x) = \prod_i P(x_i \mid \text{parents}(x_i))$. A posterior $P(Q \mid e)$ is obtained by summing joint rows that match query Q and evidence e , then normalizing by $P(e)$:

$$P(Q \mid e) = \frac{P(Q, e)}{P(e)} = \frac{\sum_y P(Q, e, y)}{\sum_{q,y} P(q, e, y)}$$

2. Task A – PEAS Description

PEAS Element	Traffic-Specific Description
Performance Measure	Accurate accident/signal-failure prediction, low false-alarm and missed-event rates, fast alerting, reliable operation with partial/noisy observations, and support for reduced response time and congestion impact.
Environment	Roads, intersections, vehicles, pedestrians, traffic signals, congestion, weather, incidents, emergency responders, and the traffic-control infrastructure. Dynamic, stochastic, and partially observable.
Actuators	Issue accident/failure alerts, notify traffic-control or emergency services, trigger operator warnings, support route advisories, and provide information for signal-control decisions.
Sensors	Roadside cameras, traffic detectors, speed/flow sensors, signal-health sensors, GPS/probe feeds, weather feeds, congestion indicators, and emergency/incident call information.

3. Bayesian Network Structures

- **Scenario 1:** $A \rightarrow D, A \rightarrow E \implies P(A, D, E) = P(A) \times P(D \mid A) \times P(E \mid A)$
- **Scenario 2:** $S \rightarrow C, S \rightarrow R \implies P(S, C, R) = P(S) \times P(C \mid S) \times P(R \mid S)$

4. Task B – Scenario 1: Road Accident Prediction

Input probabilities: $P(A) = 0.07$, $P(D \mid A) = 0.88$, $P(D \mid \neg A) = 0.25$, $P(E \mid A) = 0.82$, $P(E \mid \neg A) = 0.10$. Since D and E share parent A without a direct edge between them, D and E are conditionally independent given A .

4.1 Complete Joint Probability Table

A	D	E	Probability	Calculation Factorization
T	T	T	0.0505	$P(A) \times P(D \mid A) \times P(E \mid A) = 0.07 \times 0.88 \times 0.82 = 0.050512$
T	T	F	0.0111	$0.07 \times 0.88 \times (1 - 0.82) = 0.011088$
T	F	T	0.0069	$0.07 \times (1 - 0.88) \times 0.82 = 0.006888$
T	F	F	0.0015	$0.07 \times (1 - 0.88) \times (1 - 0.82) = 0.001512$
F	T	T	0.0233	$(1 - 0.07) \times 0.25 \times 0.10 = 0.023250$
F	T	F	0.2093	$(1 - 0.07) \times 0.25 \times (1 - 0.10) = 0.209250$
F	F	T	0.0698	$(1 - 0.07) \times (1 - 0.25) \times 0.10 = 0.069750$
F	F	F	0.6278	$(1 - 0.07) \times (1 - 0.25) \times (1 - 0.10) = 0.627750$

Note: The eight joint probabilities sum to 1.0000 (exact sum before displayed 4-decimal rounding).

4.2 Posterior Inference

Query	Inference Computation	Result
$P(A D)$	$P(D) = 0.07(0.88) + 0.93(0.25) = 0.2941$; $P(A, D)/P(D) = 0.0616/0.2941$	0.2095
$P(A E)$	$P(E) = 0.07(0.82) + 0.93(0.10) = 0.1504$; $P(A, E)/P(E) = 0.0574/0.1504$	0.3816
$P(A D, E)$	$0.050512/[0.050512 + 0.93(0.25)(0.10)] = 0.050512/0.073762$	0.6848

Interpretation: The prior accident probability is 7%. A traffic delay raises it to 20.95%, an emergency call raises it to 38.16%, and observing both raises it to 68.48%. Multiple consistent observations propagate evidence toward the common cause.

4.3 Independence Checks

- **Marginal independence fails:** $P(D) = 0.2941$ and $P(E) = 0.1504 \implies P(D)P(E) = 0.044233$, whereas actual $P(D, E) = 0.073762$. Therefore, D and E are not independent without evidence.
- **Conditional independence holds given A :** $P(D, E | A) = 0.88 \times 0.82 = 0.7216$ and $P(D, E | \neg A) = 0.25 \times 0.10 = 0.025$. These equal the products of conditional marginals, so $D \perp\!\!\!\perp E | A$.

4.4 Reusable Exact-Inference Method

The program enumerates all 2^n complete assignments, computes each joint row from Bayesian factorization, sums rows matching evidence, and returns $P(Q | e) = P(Q, e)/P(e)$. The same query engine is reused for every posterior and both scenarios, avoiding query-specific hard-coded formulas.

5. Task B – Scenario 2: Traffic Signal Failure Detection

Input probabilities: $P(S) = 0.05$, $P(C | S) = 0.90$, $P(C | \neg S) = 0.12$, $P(R | S) = 0.87$, $P(R | \neg S) = 0.08$. Camera Alert and Sensor Alert are separate channels sharing hidden cause S . Factorization: $P(S, C, R) = P(S) \times P(C | S) \times P(R | S)$.

5.1 Posterior Inference

Query	Inference Computation	Result
$P(S C)$	$P(C) = 0.05(0.90) + 0.95(0.12) = 0.1590$; $P(S, C)/P(C) = 0.0450/0.1590$	0.2830
$P(S R)$	$P(R) = 0.05(0.87) + 0.95(0.08) = 0.1195$; $P(S, R)/P(R) = 0.0435/0.1195$	0.3640
$P(S C, R)$	$0.05(0.90)(0.87)/[0.03915 + 0.95(0.12)(0.08)] = 0.03915/0.04827$	0.8111

Interpretation: Prior signal failure probability is 5%. Camera alert raises it to 28.30%, sensor alert raises it to 36.40%, and both alerts together raise it to 81.11%.

5.2 Independence Checks

- **Without evidence, C and R are not independent:** $P(C) = 0.159$, $P(R) = 0.1195 \implies P(C)P(R) = 0.01900$, whereas $P(C, R) = 0.03915 + 0.00912 = 0.04827$.
- **Given S , they are conditionally independent:** $P(C, R | S) = 0.90 \times 0.87 = 0.7830$, and $P(C, R | \neg S) = 0.12 \times 0.08 = 0.0096$. Hence, $C \perp\!\!\!\perp R | S$.

6. Implementation Design & Data Structure Specifics

The system is implemented in modular Python (`aci_assignment2_ps12.py`) without third-party dependencies, adhering strictly to object-oriented probabilistic modeling:

- **Network Topology Storage:** Variable order is stored as an immutable `tuple[str]` (e.g., `("A", "D", "E")`), preserving topological node evaluation. Parent dependencies are stored in a dictionary `dict[str, tuple[str, ...]]` mapping node names to parent variable tuples (e.g., `{"D": ("A",)}`).
- **CPTs & Parameter Evaluation:** Input probabilities parsed from `inputPS12.txt` are stored in nested dictionaries `dict[str, float]`. CPTs are dynamically encapsulated inside closure functions `true_prob_fn`

- (specifically `p_true(node, assignment)`), evaluating $P(\text{node} = \text{True} \mid \text{parents})$ on demand using conditional branch logic.
- **Variable Assignments & Evidence Mapping:** Variable state combinations and observed evidence are represented using boolean dictionaries `dict[str, bool]` mapping variable names to truth values (e.g., `{"A": True, "D": True}`).
 - **Joint Enumeration Table Data Structure:** Complete joint probability distributions (2^N rows) are generated via `itertools.product((True, False), repeat=len(self.nodes))` and stored as a `list[tuple[dict[str, bool], float]]`, pairing full variable assignment maps with their calculated joint probability float values.

6.1 Module Architecture & Execution Pipeline

Component Function /	Module Responsibility & Behavior
Input Parser	Strictly validates PS12 header, scenario identifiers, and key-value lines into structured dictionaries.
Validation	Rejects missing keys, duplicate keys, malformed syntax, invalid identifiers, and out-of-bound probabilities $([0, 1])$.
Network Model	<code>BinaryBayesianNetwork</code> class encapsulates nodes, topological parent mapping, and CPT evaluation closure.
Joint Enumeration	Systematically enumerates all assignments, applies local factorizations, and builds exact joint table.
Inference Engine	Computes marginal evidence $P(e)$ and posterior $P(Q \mid e)$ via matching row summation and normalization.
Output Formatter	Formats 4-decimal rounded output tables, posteriors, and independence test results into <code>outputPS12.txt</code> .

7. Complexity of the Implemented Method

For n binary variables, complete joint table enumeration examines 2^n assignments.

- **Time Complexity:** $\mathcal{O}(n \cdot 2^n)$ to evaluate local conditional probabilities for all 2^n assignments.
- **Space Complexity:** $\mathcal{O}(2^n)$ to store the joint probability rows in memory.

For $n = 3$ in both scenario networks, only $2^3 = 8$ assignments are evaluated per network, making exact inference computationally negligible, exact, and fully transparent.

8. Task C – Bayesian Network vs Deterministic Rule-Based Monitoring

Aspect	Bayesian Network (Probabilistic)	Deterministic Rule-Based System
Reasoning Model	Represents graded uncertainty with prior, conditional, and posterior probabilities.	Fixed IF-THEN rules producing binary or discrete output actions.
Incomplete Evidence	Handles missing data by marginalizing unobserved variables seamlessly.	Fails or requires explicit rule combinations for every missing attribute.
Overlapping Symptoms	Shared causes represent dependencies explicitly, avoiding double-counting evidence.	Multiple rules treat correlated symptoms as independent facts, biasing results.
Evidence Propagation	New observations update posteriors across connected nodes continuously.	Activates/deactivates static rules; confidence does not naturally update.
Conditional Dependence	Graph structure reflects physical dependencies accurately.	Dependencies are implicit in rule authoring, leading to inconsistencies.
Complexity Reduction	Conditional independence factors joint table into small local CPTs.	Rules grow exponentially; difficult to maintain, verify, or scale.

9. Conditional Independence, d-Separation, and Accuracy

In Scenario 1, the path $D \leftarrow A \rightarrow E$ is active when A is unobserved, causing marginal dependence between D and E . Conditioning on A blocks this fork, making D and E d-separated and conditionally independent. The same applies to $C \leftarrow S \rightarrow R$.

This structural factorization improves inference accuracy by respecting true common-cause mechanics without introducing spurious direct edges between child observations. Furthermore, parameterization is significantly reduced: an unconstrained n -variable binary joint distribution requires $2^n - 1$ parameters, whereas a factored network requires only local parent-conditioned probabilities.

10. Alternate Modeling Approach & Performance Implications

- **Alternative – Dynamic Bayesian Network (DBN):** Extends static BN across temporal time slices:

$$\text{Accident}(t) \rightarrow \text{Accident}(t+1), \quad \text{Delay}(t) \rightarrow \text{Delay}(t+1)$$

This models physical persistence and evolving sensor evidence over continuous time intervals.

- **Performance Trade-off:** DBNs provide superior real-time traffic tracking but increase state complexity. Unrolled exact enumeration grows exponentially with time steps. Practical real-time deployments utilize approximate inference methods like Particle Filtering or Variable Elimination.

11. Verification and Test Strategy

- **Happy-Path Verification:** Computed posteriors match exact expected values ($P(A | D) = 0.2095$, $P(A | E) = 0.3816$, $P(A | D, E) = 0.6848$, $P(S | C) = 0.2830$, $P(S | R) = 0.3640$, $P(S | C, R) = 0.8111$).
- **Generalization Test:** Modifying probability parameters in `inputPS12.txt` automatically yields correct updated joint tables and posteriors without code modification.
- **Negative Input Tests:** Robust error handling correctly catches missing keys, duplicate keys, invalid probability ranges (< 0 or > 1), malformed lines, and invalid scenario headers.
- **Submission Behavior:** Writes formatted output directly to `outputPS12.txt` while operating as a self-contained modular Python script.

12. Conclusion

The implemented solution satisfies all Bayesian Network topology requirements and exact-inference specifications. It correctly models common-cause dependencies, demonstrates exact conditional independence behavior, and leverages clean Python data structures to execute efficient probabilistic inference.